L06 — Spark performance: shuffle, join, skew, AQE, partitioning, cache

Môi trường: Spark 4.1.3 `local[4]`, driver 2 GB. Dữ liệu: taxi 2024-01 (2.96 triệu dòng) và một fact tổng hợp 10 triệu dòng, trong đó 60 % có key = 0. Script được chạy **2 lần**; báo cáo dùng lần 2.

File nộp

File	Nội dung
`l06_experiments.py`	TODO 1–4: E1 AQE on, E2 hint `broadcast()`, E3 salting và AQE skew join
`results.csv`	34 dòng (2 lần chạy × 17 variant)
`RESULTS.md`	bảng lần 2, nhận xét E1–E5, khuyến nghị cho cluster 20 node
`01_run1.txt`, `02_run2.txt`	log console
`03_e3_task_quantiles.txt`	phân vị thời gian task và số bản ghi shuffle/task (REST API)

src/l06_experiments.py (TODO 1-4)

```
    run("E1", "AQE off, shuffle.partitions=200", q())
    set_conf(spark__sql__shuffle__partitions=8)
    run("E1", "AQE off, shuffle.partitions=8", q())
    set_conf(spark__sql__shuffle__partitions=1)
    run("E1", "AQE off, shuffle.partitions=1", q())
    # TODO 1: turn AQE back ON (keep shuffle.partitions=200) and run the same query as
    #         variant "AQE on,  shuffle.partitions=200". What does AQEShuffleRead do?
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__shuffle__partitions=200)
    run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small shuffle partitions")
def e2_broadcast_vs_smj():
    q = lambda: (trips_df().join(zones_df(), F.col("PULocationID") == F.col("LocationID"))
                 .groupBy("Borough").agg(F.count("*").alias("trips")))
    reset_conf()
    set_conf(spark__sql__adaptive__enabled=False)
    run("E2", "AQE off, broadcast (threshold 10MB)", q())
    set_conf(spark__sql__autoBroadcastJoinThreshold=-1)
    run("E2", "AQE off, sort-merge (threshold -1)", q())
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__autoBroadcastJoinThreshold=-1)
    # TODO 2: with AQE on and the threshold still -1, force a broadcast join with the F.broadcast()
    #         hint on zones_df() and record it as variant "AQE on,  threshold -1 + broadcast() hint".
    run("E2", "AQE on,  threshold -1 + broadcast() hint",
        trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") == F.col("LocationID"))
        .groupBy("Borough").agg(F.count("*").alias("trips")), "an explicit hint beats the threshold")
def skew_inputs():
    fact = (spark.range(N_FACT)
            .withColumn("key", F.when(F.rand(seed=7) < HOT_SHARE, F.lit(0))
                        .otherwise((F.rand(seed=11) * N_KEYS).cast("long")))
            .withColumn("amount", (F.rand(seed=3) * 100).cast("double")))
    dim = (spark.range(N_KEYS).withColumnRenamed("id", "key")
           .withColumn("segment", F.concat(F.lit("seg_"), (F.col("key") % 50).cast("string"))))
    return fact, dim
def e3_skew():
    reset_conf()
    fact, dim = skew_inputs()
    fact.groupBy((F.col("key") == 0).alias("is_hot_key")).count().show()
    # force a shuffle join (dim is 200k rows -> could be broadcast, which would hide the skew)
    set_conf(spark__sql__autoBroadcastJoinThreshold=-1, spark__sql__adaptive__enabled=False,
             spark__sql__shuffle__partitions=16)
    joined = fact.join(dim, "key").groupBy("segment").agg(F.sum("amount").alias("amount"))
    base = run("E3", "skewed SMJ, AQE off", joined)
    # Fix 1: SALTING -- spread the hot key over SALT sub-keys. Only the HOT key is salted (random
    # salt 0..SALT-1); every other key keeps salt 0. The dimension row of the hot key is replicated
    # SALT times, all other dimension rows once -> almost no extra data.
    SALT = 64   # >> shuffle partitions, so hash collisions of (key, salt) even out
    # TODO 3: SALTING of the hot key only.
    #   fact_s: new int column "salt" = random 0..SALT-1 when key == 0, else 0
    #   dim_s : dim rows with key != 0 get salt 0; the dim row with key == 0 is cross-joined with
    #           spark.range(SALT) (as int column "salt") -> SALT copies
    #   salted: fact_s.join(dim_s, ["key", "salt"]) then the same groupBy/agg as `joined`
    hot = F.col("key") == 0
    fact_s = fact.withColumn("salt", F.when(hot, (F.rand(seed=5) * SALT).cast("int")).otherwise(F.lit(0)))
    salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
    dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
             .unionByName(dim.filter(hot).crossJoin(salts)))
    salted = fact_s.join(dim_s, ["key", "salt"]).groupBy("segment").agg(F.sum("amount").alias("amount"))
    run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
    # Fix 2: AQE skew join -- split the skewed shuffle partition at runtime
    # TODO 4: enable AQE + skew-join handling. Our data is small, so lower the thresholds:
    #         advisoryPartitionSizeInBytes=4m, skewJoin.skewedPartitionThresholdInBytes=16m,
    #         skewJoin.skewedPartitionFactor=3, and disable coalescePartitions for a clean comparison.
    set_conf(spark__sql__adaptive__enabled=True, spark__sql__adaptive__skewJoin__enabled=True,
             spark__sql__adaptive__coalescePartitions__enabled=False,
             spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
             spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
             spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
```

Hình 1: code

Bảng kết quả (lần 2)

```
spark-submit l06_experiments.py (run 2)

Spark 4.1.3 | cores 4 | input ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']

===== E1: e1_shuffle_partitions
[E1] AQE off, shuffle.partitions=200           2.20 s | tasks=204 skewiest stage: median=6.0 ms max=60.0 ms | Exchange
[E1] AQE off, shuffle.partitions=8             0.33 s | tasks=12 skewiest stage: median=177.0 ms max=178.0 ms | Exchange
[E1] AQE off, shuffle.partitions=1             0.44 s | tasks=5 skewiest stage: median=232.0 ms max=233.0 ms | Exchange
[E1] AQE on,  shuffle.partitions=200           0.53 s | tasks=5 skewiest stage: median=271.0 ms max=274.0 ms | AQEShuffleRead Exchange

===== E2: e2_broadcast_vs_smj
[E2] AQE off, broadcast (threshold 10MB)       1.20 s | tasks=13 skewiest stage: median=626.0 ms max=635.0 ms | BroadcastExchange BroadcastHashJoin
  Exchange
[E2] AQE off, sort-merge (threshold -1)        1.51 s | tasks=21 skewiest stage: median=386.0 ms max=591.0 ms | Exchange SortMergeJoin
[E2] AQE on,  threshold -1 + broadcast() hint  0.40 s | tasks=6 skewiest stage: median=157.0 ms max=168.0 ms | AQEShuffleRead BroadcastExchange
  BroadcastHashJoin Exchange

===== E3: e3_skew
+----------+-------+
|is_hot_key|  count|
+----------+-------+
|      true|5998366|
|     false|4001634|
+----------+-------+

[E3] skewed SMJ, AQE off                       4.10 s | tasks=40 skewiest stage: median=165.0 ms max=2366.0 ms | Exchange SortMergeJoin
[E3] salted SMJ (salt=64), AQE off             4.01 s | tasks=56 skewiest stage: median=590.0 ms max=1197.0 ms | Exchange SortMergeJoin
[E3] skewed SMJ, AQE skewJoin on               2.22 s | tasks=27 skewiest stage: median=186.0 ms max=682.0 ms | AQEShuffleRead Exchange
  SortMergeJoin(skew=true)
same result with and without salting: True (500061629.0)

===== E4: e4_repartition_vs_coalesce
[E4] repartition(64)                          7.25 s | files=64 | tasks=68 | Exchange
[E4] repartition(4)                           4.40 s | files=4 | tasks=8 | Exchange
[E4] coalesce(4)                              3.12 s | files=3 | tasks=4 | Coalesce
[E4] coalesce(1)                              6.95 s | files=1 | tasks=1 | Coalesce

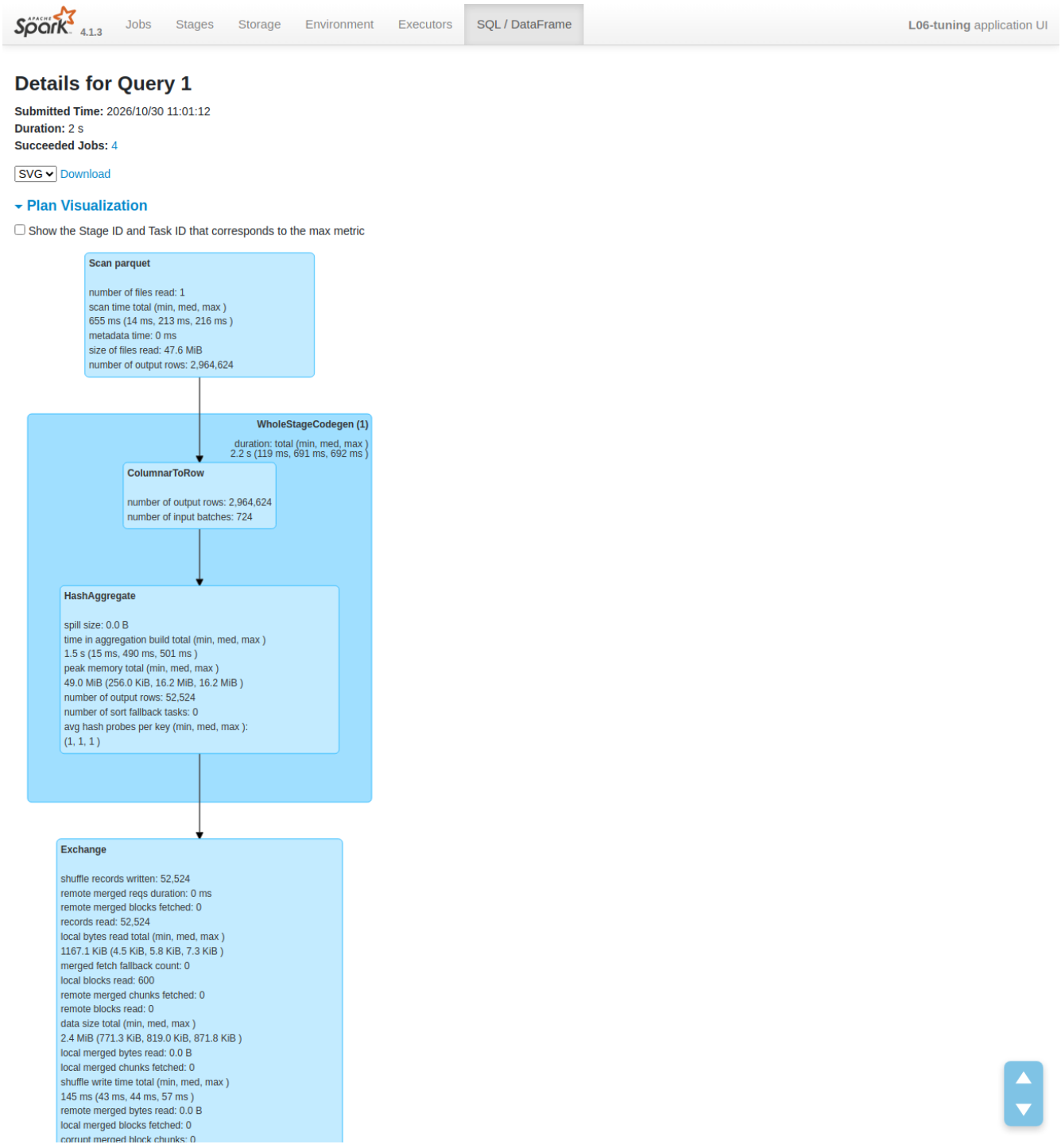
===== E5: e5_cache
[E5] no cache                                 6.40 s
[E5] with cache (MEMORY_AND_DISK)             0.99 s
```

```
results table (run 2)

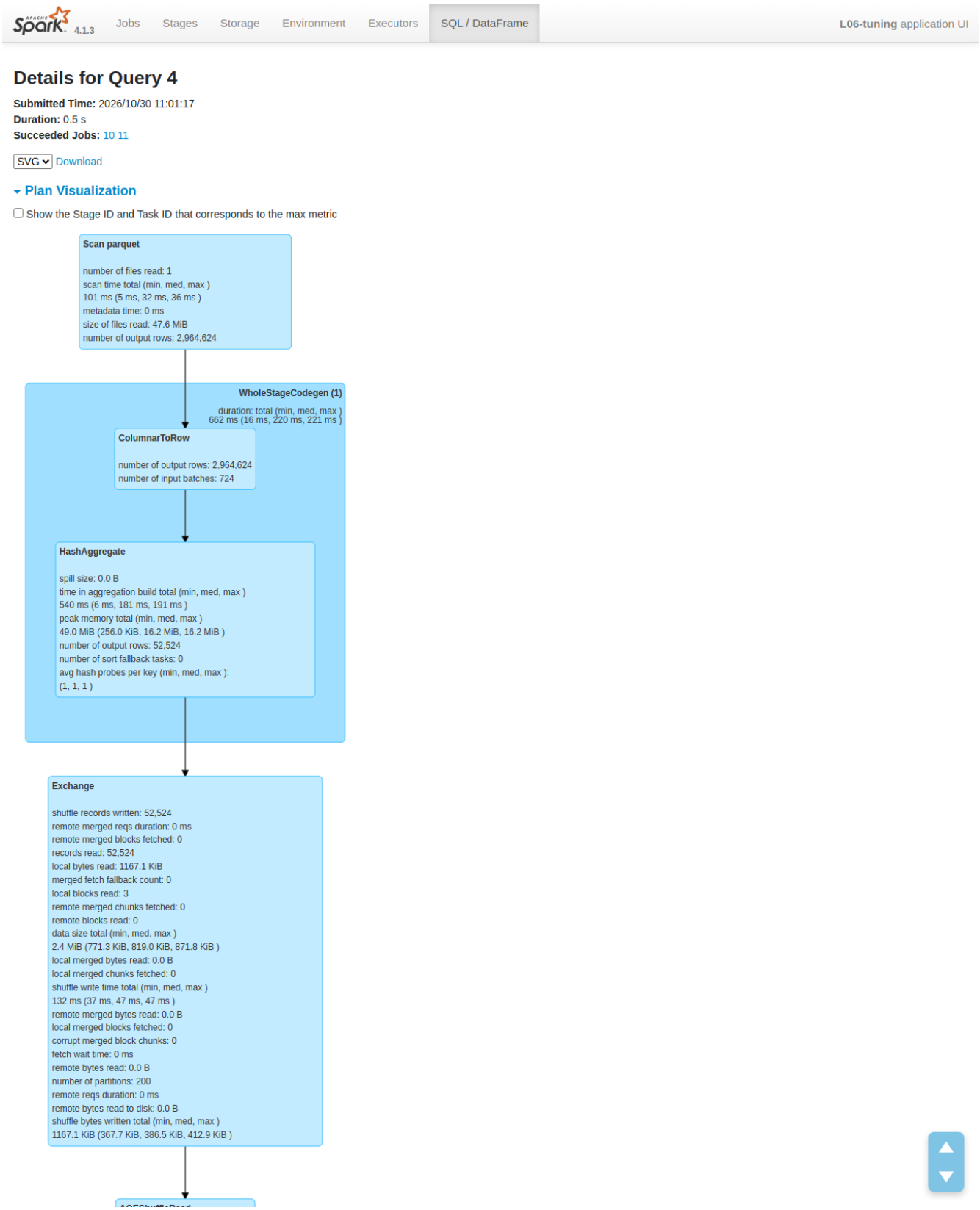
 experiment | variant | seconds | total tasks | task median ms* | task max ms* | plan (final, after AQE)
---|---|---|---|---|---|---
 E1 | AQE off, shuffle.partitions=200 | 2.2 | 204 | 6.0 | 60.0 | Exchange
 E1 | AQE off, shuffle.partitions=8 | 0.33 | 12 | 177.0 | 178.0 | Exchange
 E1 | AQE off, shuffle.partitions=1 | 0.44 | 5 | 232.0 | 233.0 | Exchange
 E1 | AQE on,  shuffle.partitions=200 | 0.53 | 5 | 271.0 | 274.0 | AQEShuffleRead Exchange
 E2 | AQE off, broadcast (threshold 10MB) | 1.2 | 13 | 626.0 | 635.0 | BroadcastExchange BroadcastHashJoin Exchange
 E2 | AQE off, sort-merge (threshold -1) | 1.51 | 21 | 386.0 | 591.0 | Exchange SortMergeJoin
 E2 | AQE on,  threshold -1 + broadcast() hint | 0.4 | 6 | 157.0 | 168.0 | AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange
 E3 | skewed SMJ, AQE off | 4.1 | 40 | 165.0 | 2366.0 | Exchange SortMergeJoin
 E3 | salted SMJ (salt=64), AQE off | 4.01 | 56 | 590.0 | 1197.0 | Exchange SortMergeJoin
 E3 | skewed SMJ, AQE skewJoin on | 2.22 | 27 | 186.0 | 682.0 | AQEShuffleRead Exchange SortMergeJoin(skew=true)
 E4 | repartition(64) | 7.25 | 68 | 152.0 | 980.0 | Exchange
 E4 | repartition(4) | 4.4 | 8 | 2185.0 | 2210.0 | Exchange
 E4 | coalesce(4) | 3.12 | 4 | 2907.0 | 2970.0 | Coalesce
 E4 | coalesce(1) | 6.95 | 1 | 6800.0 | 6800.0 | Coalesce
 E5 | no cache | 6.4 |  |  |  |
 E5 | cache materialisation (count) | 3.42 |  |  |  |
 E5 | with cache (MEMORY_AND_DISK) | 0.99 |  |  |  |
* median / max task run time of the most skewed stage of that query
```

Bảng đầy đủ, nhận xét từng thí nghiệm và khuyến nghị nằm trong `RESULTS.md`.

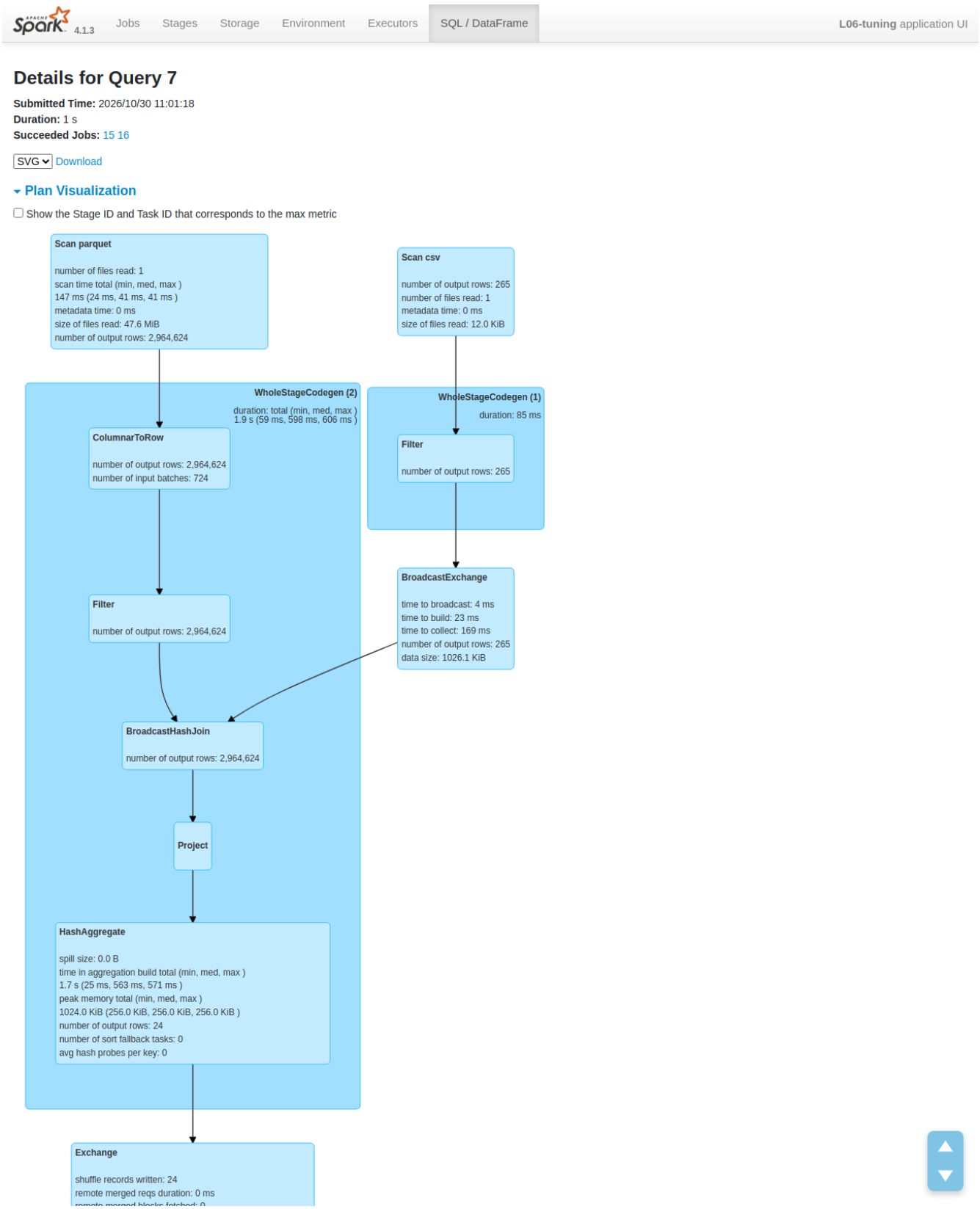
Bằng chứng trên Spark UI



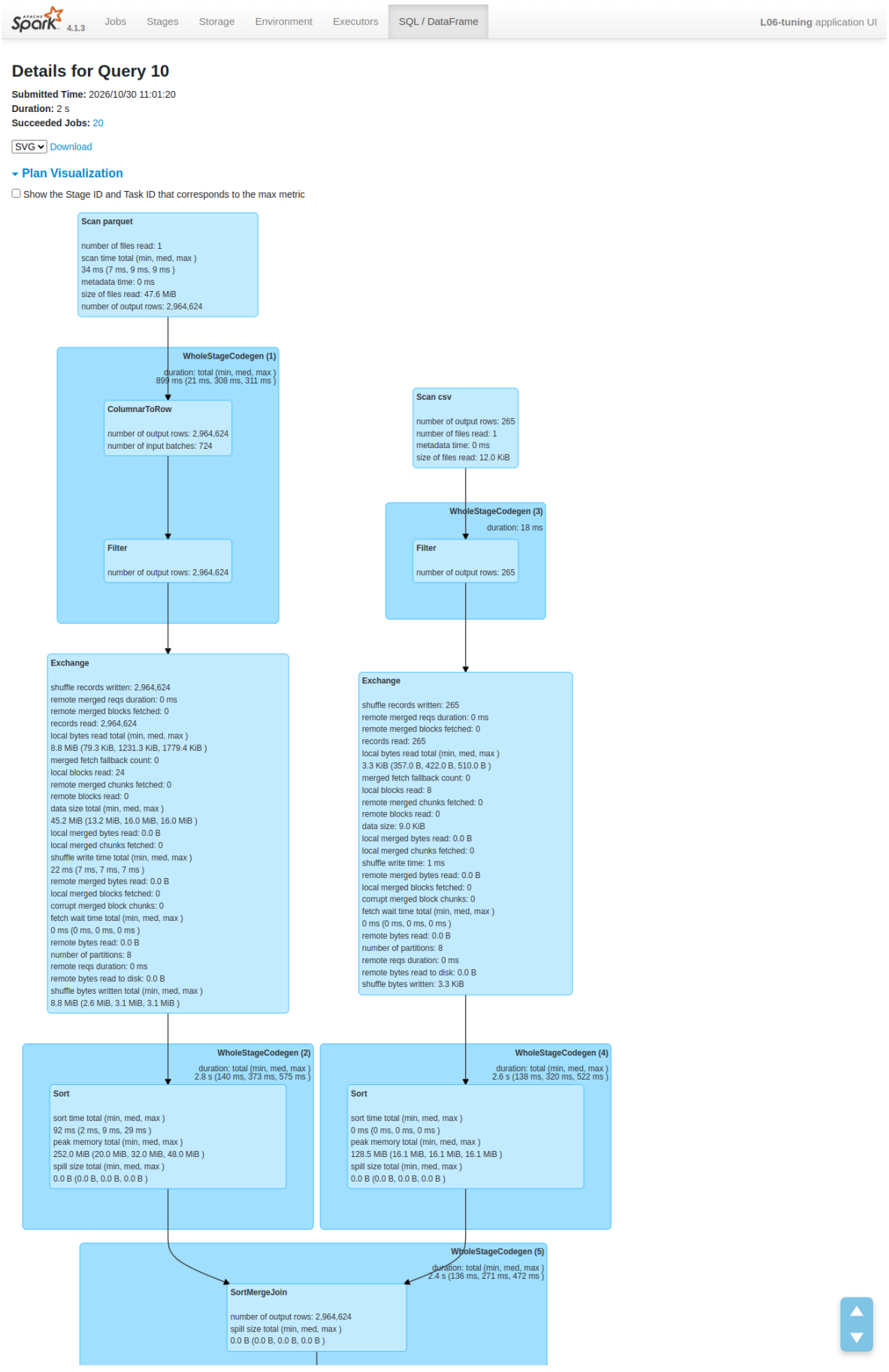
Hình 2: **E1**: AQE off, 200 partition → `Exchange` 200 partition



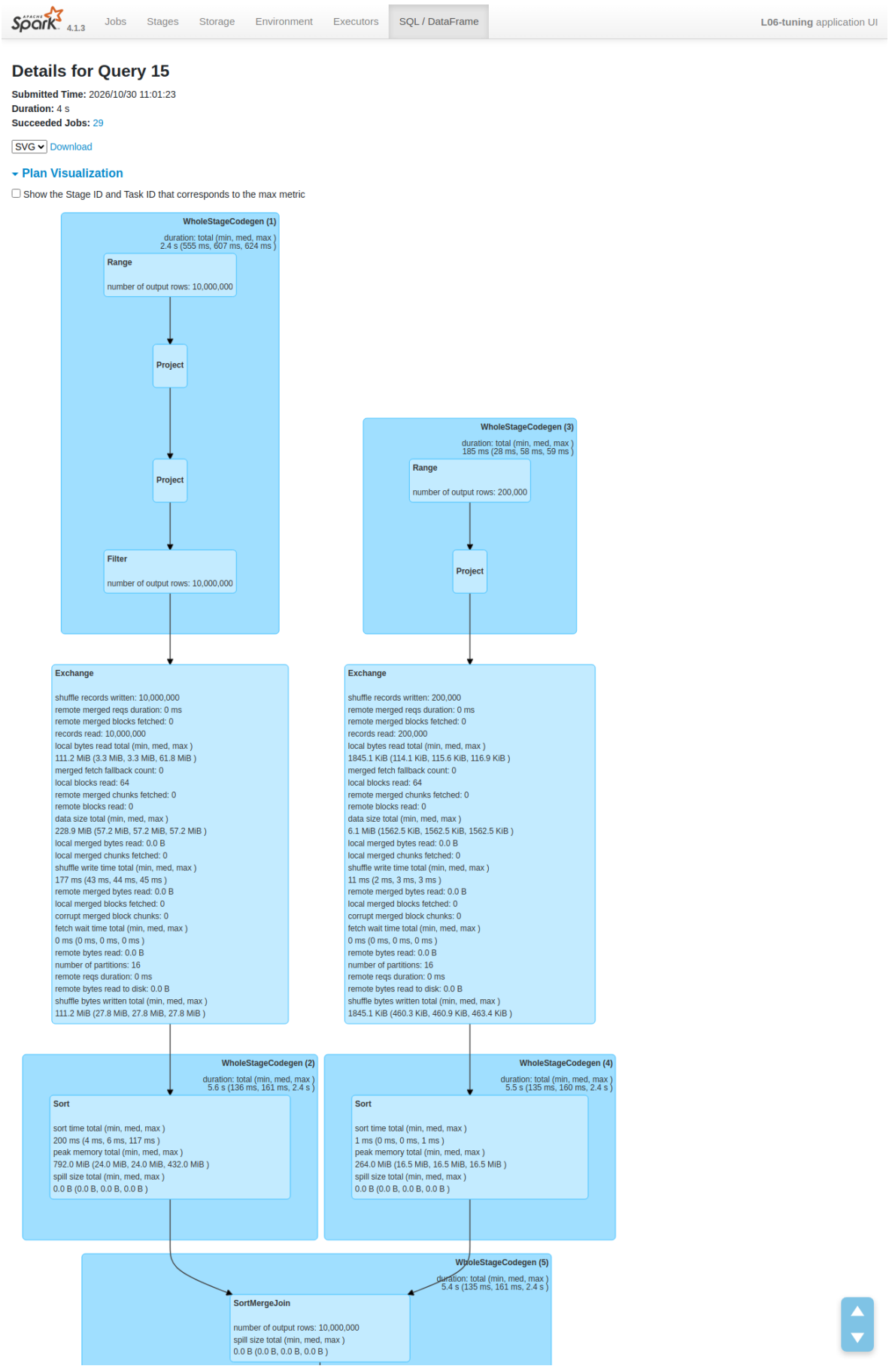
Hình 3: **E1**: AQE on → `AQEShuffleRead` coalesced



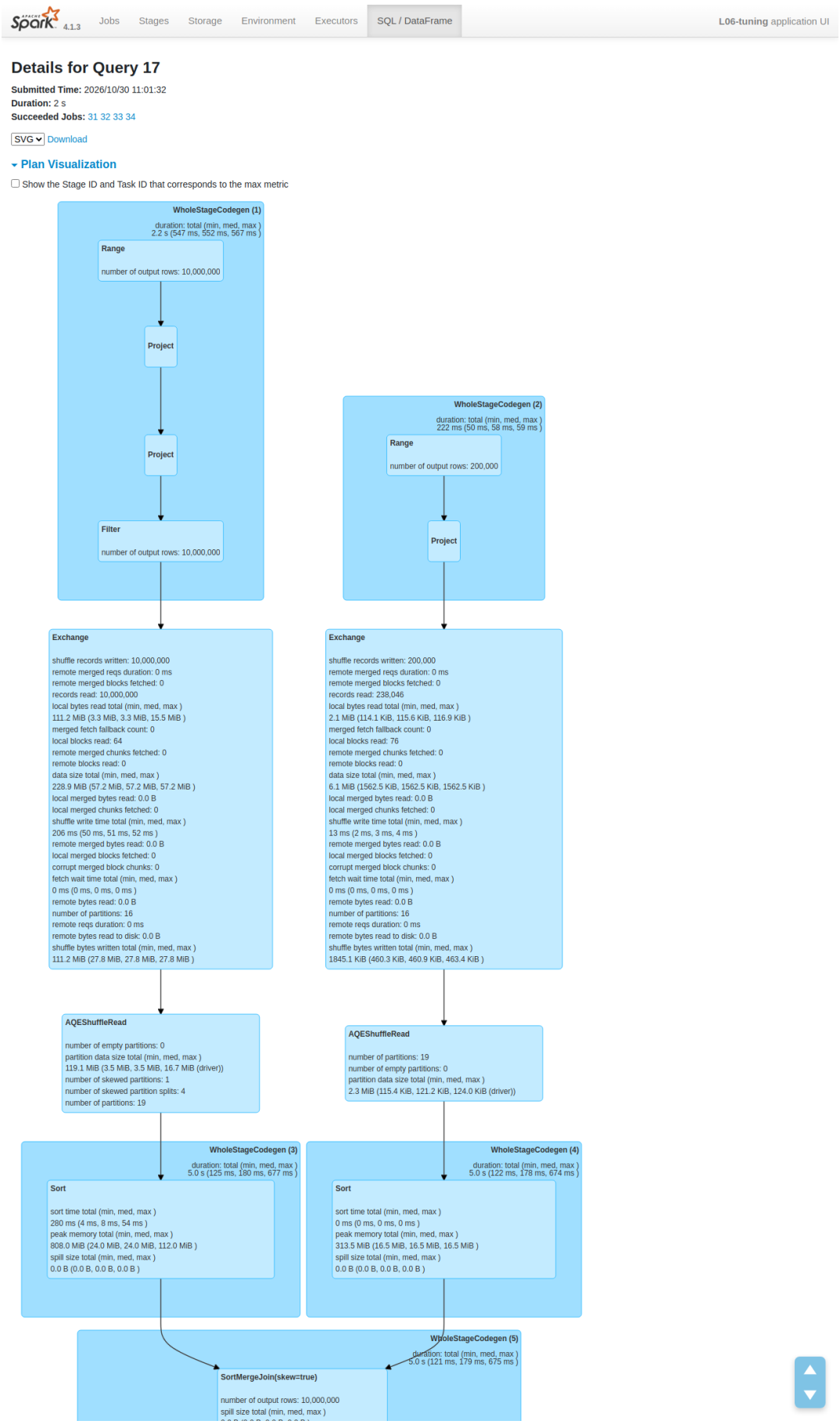
Hình 4: **E2**: `BroadcastExchange` + `BroadcastHashJoin`



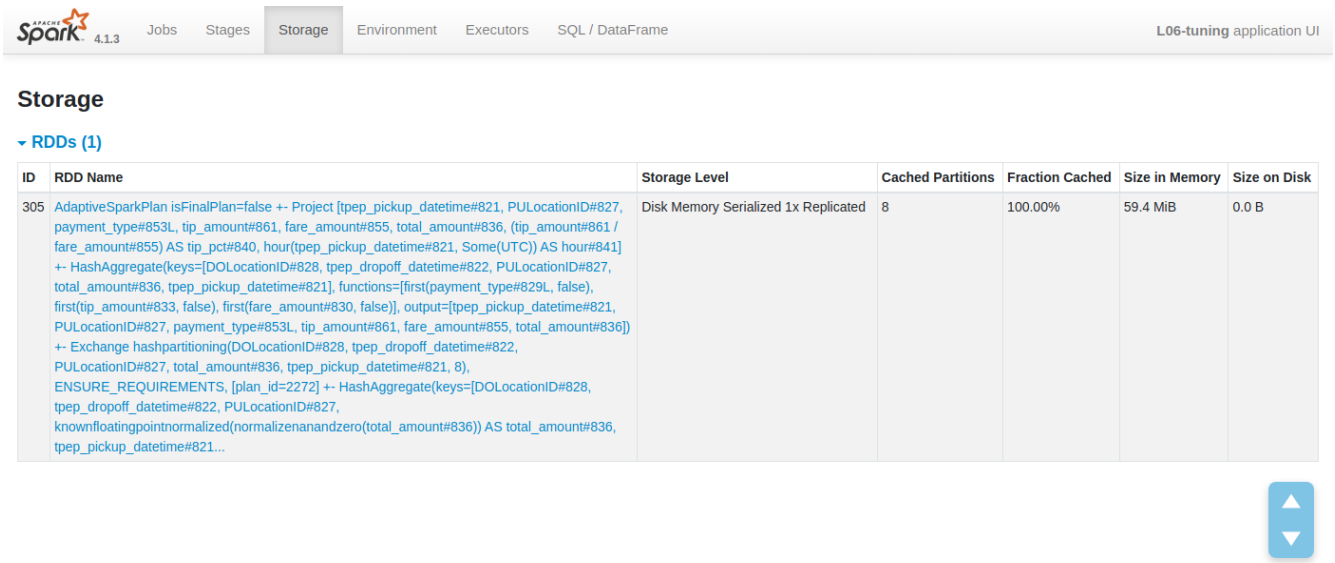
Hình 5: **E2**: threshold -1 → `Exchange` + `Sort` hai phía + `SortMergeJoin`



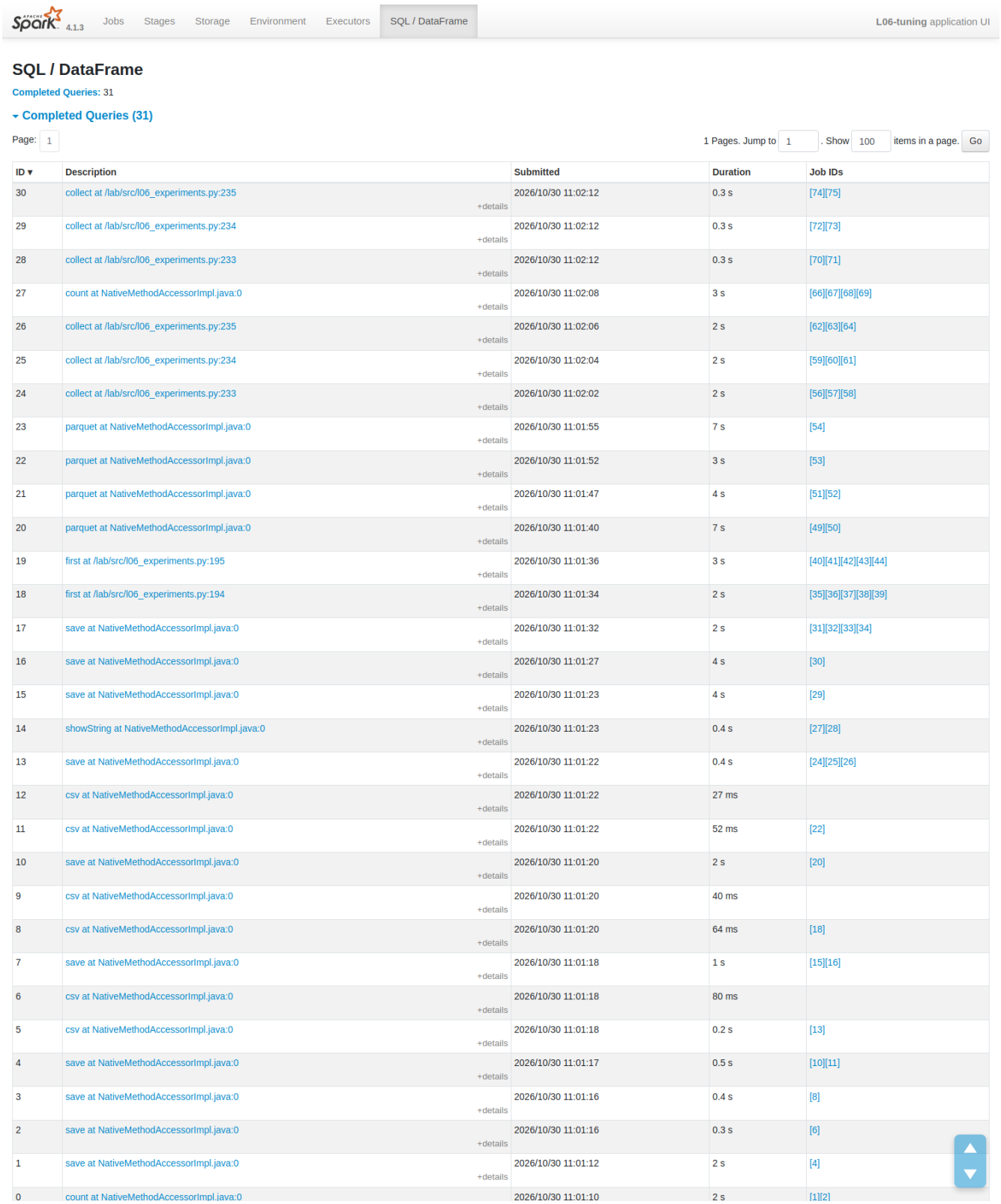
Hình 6: **E3**: baseline SMJ



Hình 7: **E3**: AQE → `AQEShuffleRead` (skewed partitions: 1, splits: 4, partitions: 19) + `SortMergeJoin(skew=true)`



Hình 8: **E5**: Storage tab, `cleaned` cache 8/8 partition, 100 %, 59.4 MiB



Hình 9: Danh sách SQL query

Skew: Summary Metrics của stage join

Stage	Task time min / median / max	Shuffle records median / max
42: baseline, AQE off	140 / 165 / **2,366 ms**	262,619 / **6,264,438** (61.9 MiB, so với 3.4 MiB)
46: salting 64	308 / 590 / 1,197 ms	638,133 / 917,267
52: AQE skew join	129 / 186 / 682 ms	262,677 / 1,576,365 (≈ 6.26 M / 4)

E3: task time and shuffle records per task

```
# task quantiles [min, p25, median, p75, max] from /api/v1/.../taskSummary
stage 42: runTime [140.0, 157.0, 165.0, 379.0, 2366.0] shuffleReadRecords [260166.0, 261225.0, 262619.0, 264416.0, 6264438.0]
stage 46: runTime [308.0, 484.0, 590.0, 901.0, 1197.0] shuffleReadRecords [359285.0, 543450.0, 638133.0, 823338.0, 917267.0]
stage 52: runTime [129.0, 144.0, 186.0, 199.0, 682.0] shuffleReadRecords [260166.0, 261225.0, 262677.0, 265121.0, 1576365.0]
```

Details for Stage 42 (Attempt 0)

Submitted: 2026/10/30 11:01:24
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 6 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 113.0 MiB / 10200000
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 29

- DAG Visualization
- Show Additional Metrics
- Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.2 s	0.2 s	0.4 s	2 s
GC Time	0.0 ms	3.0 ms	3.0 ms	6.0 ms	25.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262619	3.4 MiB / 264416	61.9 MiB / 6264438
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

- Aggregated Metrics by Executor

Tasks (16)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	298	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 265121	
1	299	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 264416	
2	300	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 260569	
3	301	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 262353	
4	302	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	303	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	304	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	305	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	306	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260412	
9	307	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261225	
10	308	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262619	
11	309	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 264108	
12	310	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262023	
13	311	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	2 s	25.0 ms	1.7 KiB / 50	61.9 MiB / 6264438	
14	312	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.1 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
15	313	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 16 of 16 entries Previous 1 Next

Details for Stage 46 (Attempt 0)

Submitted: 2026/10/30 11:01:29
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 11 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 127.0 MiB / 10200063
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 30

- DAG Visualization
- Show Additional Metrics
- Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.3 s	0.5 s	0.6 s	0.9 s	1 s
GC Time	0.0 ms	0.0 ms	5.0 ms	7.0 ms	12.0 ms
Shuffle Read Size / Records	4.9 MiB / 359285	6.9 MiB / 543450	8 MiB / 638133	10 MiB / 823338	11 MiB / 917267
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

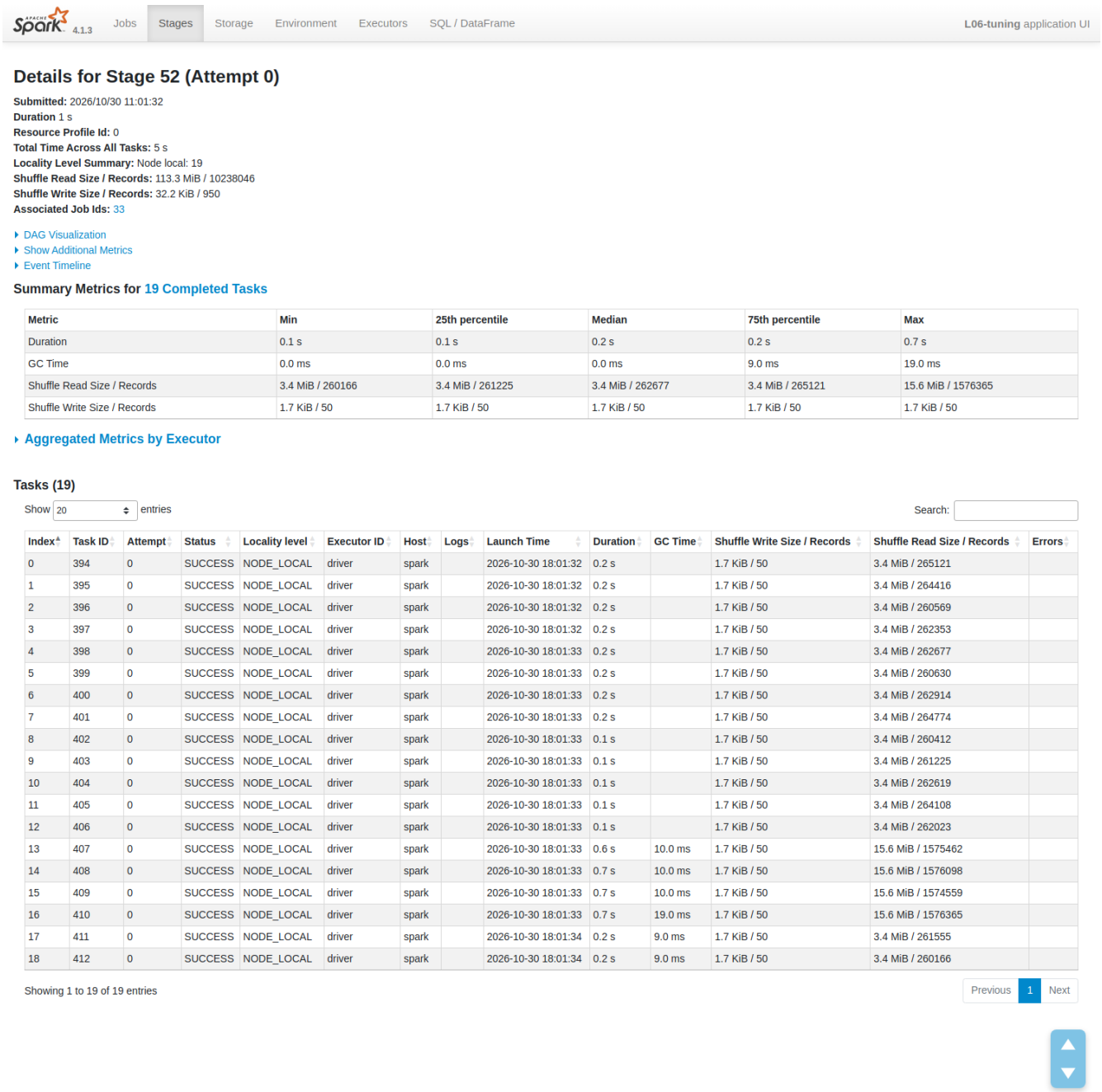
- Aggregated Metrics by Executor

Tasks (16)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	354	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1.0 s	5.0 ms	1.7 KiB / 50	9 MiB / 733891	
1	355	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	0.9 s	5.0 ms	1.7 KiB / 50	7.9 MiB / 637285	
2	356	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 824727	
3	357	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 825508	
4	358	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	10.0 ms	1.7 KiB / 50	10 MiB / 823338	
5	359	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	7.0 ms	1.7 KiB / 50	8 MiB / 638553	
6	360	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	3.0 ms	1.7 KiB / 50	8 MiB / 637050	
7	361	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.4 s		1.7 KiB / 50	4.9 MiB / 359285	
8	362	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	7.0 ms	1.7 KiB / 50	11 MiB / 917267	
9	363	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.3 s	7.0 ms	1.7 KiB / 50	4.9 MiB / 361145	
10	364	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	8 MiB / 638133	
11	365	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	6.9 MiB / 543450	
12	366	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.5 s		1.7 KiB / 50	6.9 MiB / 539233	
13	367	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.6 s		1.7 KiB / 50	8.9 MiB / 727672	
14	368	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	5.9 MiB / 448281	
15	369	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	7 MiB / 545245	

Showing 1 to 16 of 16 entries Previous 1 Next



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 52 (Attempt 0)

Submitted: 2026/10/30 11:01:32
Duration 1 s
Resource Profile Id: 0
Total Time Across All Tasks: 5 s
Locality Level Summary: Node local: 19
Shuffle Read Size / Records: 113.3 MiB / 10238046
Shuffle Write Size / Records: 32.2 KiB / 950
Associated Job Ids: 33

DAG Visualization
Show Additional Metrics
Event Timeline

Summary Metrics for 19 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.1 s	0.2 s	0.2 s	0.7 s
GC Time	0.0 ms	0.0 ms	0.0 ms	9.0 ms	19.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262677	3.4 MiB / 265121	15.6 MiB / 1576365
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

Aggregated Metrics by Executor

Tasks (19)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	394	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 265121	
1	395	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 264416	
2	396	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 260569	
3	397	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 262353	
4	398	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	399	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	400	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	401	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	402	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 260412	
9	403	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 261225	
10	404	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262619	
11	405	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 264108	
12	406	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262023	
13	407	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.6 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1575462	
14	408	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1576098	
15	409	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1574559	
16	410	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	19.0 ms	1.7 KiB / 50	15.6 MiB / 1576365	
17	411	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
18	412	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 19 of 19 entries Previous 1 Next

Trả lời checkpoint

Q1. Kết quả `groupBy(PU, DO)` chỉ có vài MB, khoảng 20 nghìn nhóm. 200 partition nghĩa là 204 task, mỗi task chạy 6 ms; chi phí lập lịch, serialize và khởi tạo task áp đảo, nên mất 2.2 s so với 0.33 s khi dùng 8 partition. `shuffle.partitions=1` nhanh ở đây chỉ vì dữ liệu shuffle tí hon. Trên dữ liệu thật, một reducer phải xử lý **toàn bộ** shuffle: không có song song, spill ra đĩa, dễ OOM. Quy tắc: khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

Q2. Sau khi stage map của shuffle chạy xong, AQE có **thống kê thật** về kích thước từng partition shuffle (map output statistics). `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`. Nhờ đó 200 partition chỉ còn 1 task reduce (tổng 5 task, thay vì 204). Planner tĩnh chỉ có **ước lượng** (kích thước

file), và ước lượng thường sai sau filter, join hay aggregate.

Q3. Con số chứng minh skew là **task max** ≫ **median** ở stage join: 2,366 ms so với 165 ms (khoảng 14×). Task đó đọc 6.26 triệu bản ghi trong khi median chỉ 262 nghìn. Salting giảm straggler (max 1,197 ms) nhưng **wall-clock gần như không đổi** (4.10 → 4.01 s). Lý do: - Nó thêm việc ở mọi nơi: `rand()` cho mỗi dòng, thêm cột join, 64 bản sao dim. - 6 triệu dòng hot vẫn phải chia trên 16 partition × 4 core, nên median tăng từ 165 lên 590 ms. Với 4 core, tổng công việc là như nhau, chỉ phân bổ lại.

Salting thắng rõ khi có **nhiều core và partition**: straggler sẽ giữ hàng trăm executor ngồi chờ. Nó cũng thắng khi công việc trên mỗi dòng nặng (dòng rộng, join many-to-many, spill), hoặc khi skew nặng hơn (ví dụ 90 % hot).

Q4. Một partition bị coi là skew khi kích thước của nó **lớn hơn** **`skewedPartitionFactor` × median và** lớn hơn `skewedPartitionThresholdInBytes`. - Mặc định là 5× và 256 MB; lab hạ xuống 3× và 16 MB. - Ở đây partition hot 61.9 MiB so với median 3.4 MiB.

AQE **tách** partition đó thành nhiều task theo khoảng map output (ở đây 4 split, nên 16 → 19 partition), rồi **nhân bản** partition tương ứng của phía bên kia. Plan khi đó hiện `SortMergeJoin(skew=true)`.

Giới hạn của AQE so với salting: - chỉ áp dụng cho join có shuffle ở cả hai phía (SMJ hoặc shuffled hash); - không áp dụng cho mọi kiểu join (không tách được phía cần giữ nguyên của outer join); - cần ngưỡng hợp với dữ liệu; - **không xử lý skew trong** **`groupBy`** hay skew sau join. Salting, hay aggregate hai pha, xử lý được những trường hợp này.

Q5. Không nên broadcast dù một bên "nhỏ" khi: - bảng đó vẫn lớn trong bộ nhớ: driver phải collect trước rồi gửi bản sao tới mọi executor, gây OOM driver hoặc broadcast timeout; - thống kê **đánh giá thấp** kích thước, ví dụ sau filter hay UDF; - là **full outer join** (không hỗ trợ), hoặc phía cần broadcast là phía được giữ của outer join; - join chạy lặp lại nhiều lần trên dữ liệu thay đổi, nên mỗi lần phải dựng lại broadcast.

Q6. - `repartition(n)` **luôn shuffle**: partition cân bằng (4 file đều nhau), và stage trước giữ nguyên song song. Cái giá là thêm một shuffle (4.40 s). - `coalesce(n)` gộp partition sẵn có **không shuffle**: rẻ (3.12 s), nhưng **có thể tạo file lệch**. Ở đây chỉ ra 3 file vì input chỉ có 3 row group, nên một partition rỗng. - Vì `coalesce` là narrow dependency, nó **giảm song song của cả stage trước**: `coalesce(1)` chạy scan và ghi trong **1 task** (6,800 ms).

Q7. DataFrame được cache là kết quả của một bước **đắt** (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại 3 lần. - Không cache: dedup chạy 3 lần, 6.40 s. - Có cache: dedup chạy 1 lần (3.42 s) rồi 3 query chỉ còn 0.99 s.

Một scan Parquet thuần, đã prune cột, thì vốn rẻ: đọc vectorized, có column pruning và predicate pushdown. Cache khi đó chỉ tốn bộ nhớ và thời gian materialize, gần như không lợi. Quy tắc: cache cái **đắt và được dùng lại**, rồi `unpersist()`.

Q8. Checklist trước khi mua thêm phần cứng: 1. **Đọc plan và Spark UI**: tìm stage chậm nhất, spill, GC, và tỉ lệ task max/median để phát hiện skew. 2. **Lọc và chọn cột sớm**: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC). 3. **Kích thước partition shuffle hợp lý**: bật AQE (coalesce và skew join), khoảng 100–200 MB mỗi partition. 4. **Chọn đúng chiến lược join**: broadcast bảng nhỏ; với skew thì dùng AQE skew join, salting, hoặc tách hot key. 5. **Tránh UDF Python** (dùng hàm built-in hoặc pandas UDF), **cache có chọn lọc**, và **tránh file nhỏ** khi ghi (repartition hoặc compaction).